

QUERY PROCESSING FOR DATA SCIENCE

Question 2: Missing Data Handling

Problem Explanation

In real-world datasets, some values may be missing due to incomplete data collection or human error. Missing values can affect analysis and machine learning models. This program identifies missing values, replaces them with the average of the respective column, and displays the cleaned dataset.

CODE

```
import pandas as pd

data = {
    "Name": ["A", "B", "C", "D"],
    "Marks": [80, None, 75, 90]
}

df = pd.DataFrame(data)
print("Original Dataset")
print(df)
print("\nMissing Values")
print(df.isnull())
average = df["Marks"].mean()
df["Marks"] = df["Marks"].fillna(average)
print("\nCleaned Dataset")
print(df)
```

OUTPUT

```
QUERY1.py - C:/Users/poojashri/OneDrive/Desktop/QUERY1.py (3.13.7)
import pandas as pd
data = {
    "Name": ["A", "B", "C", "D"],
    "Marks": [80, None, 75, 90]
}
df = pd.DataFrame(data)
print("Original Dataset")
print(df)
print("\nMissing Values")
print(df.isnull())
average = df["Marks"].mean()
df["Marks"] = df["Marks"].fillna(average)
print("\nCleaned Dataset")
print(df)

IDLE Shell 3.13.7
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
Original Dataset
   Name Marks
0    A   80.0
1    B    NaN
2    C   75.0
3    D   90.0

Missing Values
   Name Marks
0  False  False
1  False   True
2  False  False
3  False  False

Cleaned Dataset
   Name Marks
0    A  80.000000
1    B  81.666667
2    C  75.000000
3    D  90.000000

>>>
```

CONCLUSION

The program successfully detects missing values and replaces them with the column average, resulting in a complete and cleaner dataset for analysis.

Question 4: Sorting Dataset

Problem Explanation

Sorting data helps identify the highest and lowest values quickly. This program sorts products based on sales amount and displays the top-selling products.

CODE

```
import pandas as pd
```

```
data = {
```

```
    "Product": ["A", "B", "C", "D", "E"],
```

```
    "Sales": [5000, 9000, 7000, 12000, 6500]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
sorted_df = df.sort_values(by="Sales", ascending=False)
```

```
print("Top Selling Products:")
```

```
print(sorted_df.head(10))
```

OUTPUT

```
QUERY1.py - C:/Users/poojashri/OneDrive/Desktop/QUERY1.py (3.13.7)
import pandas as pd
data = {
    "Product": ["A", "B", "C", "D", "E"],
    "Sales": [5000, 9000, 7000, 12000, 6500]
}
df = pd.DataFrame(data)
sorted_df = df.sort_values(by="Sales", ascending=False)
print("Top Selling Products:")
print(sorted_df.head(10))

IDLE Shell 3.13.7
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
Top Selling Products:
   Product Sales
3        D 12000
1         B  9000
2         C  7000
4         E  6500
0         A  5000

>>>
```

CONCLUSION

The program arranges products in descending order of sales, making it easy to identify the best-performing products.

Question 6: Data Cleaning

Problem Explanation

Datasets often contain extra spaces, inconsistent letter cases, and missing values. Cleaning the data improves its quality and ensures accurate analysis.

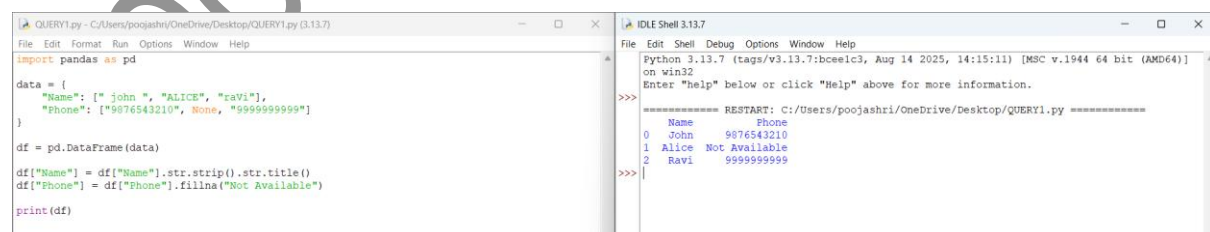
Python Code

```
import pandas as pd

data = {
    "Name": [" john ", "ALICE", "raVi"],
    "Phone": ["9876543210", None, "9999999999"]
}

df = pd.DataFrame(data)
df["Name"] = df["Name"].str.strip().str.title()
df["Phone"] = df["Phone"].fillna("Not Available")
print(df)
```

OUTPUT

The screenshot shows a Python IDE with two windows. The left window, titled 'QUERY1.py', contains the Python code for data cleaning. The right window, titled 'IDLE Shell 3.13.7', shows the output of the code. The output is a table with three rows: the first row is the header with 'Name' and 'Phone' columns; the second row is for 'John' with phone number '9876543210'; the third row is for 'Alice' with 'Not Available' in the phone column; and the fourth row is for 'Ravi' with phone number '9999999999'.

```
File Edit Format Run Options Window Help
import pandas as pd

data = {
    "Name": [" john ", "ALICE", "raVi"],
    "Phone": ["9876543210", None, "9999999999"]
}

df = pd.DataFrame(data)
df["Name"] = df["Name"].str.strip().str.title()
df["Phone"] = df["Phone"].fillna("Not Available")
print(df)
```

```
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
      Name      Phone
0   John  9876543210
1  Alice  Not Available
2   Ravi  9999999999
>>>
```

Conclusion

The dataset becomes cleaner by removing unnecessary spaces, formatting names properly, and filling missing phone numbers.

Question 7: Data Transformation

Problem Explanation

Data transformation converts data into another format for better usability. This program converts product prices from USD to INR by creating a new column.

Python Code

```
import pandas as pd

data = {
    "Product": ["Pen", "Book", "Bag"],
    "Price_USD": [2, 10, 25]
}

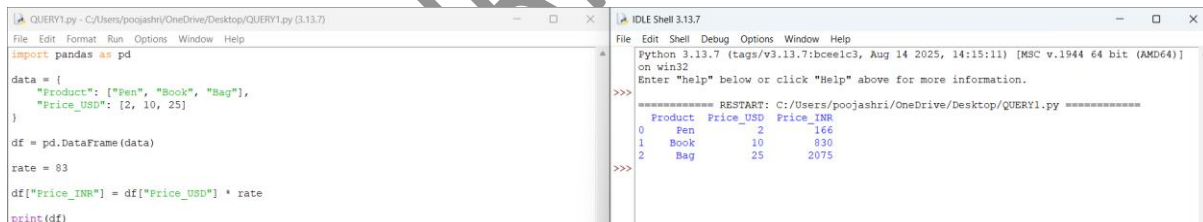
df = pd.DataFrame(data)

rate = 83

df["Price_INR"] = df["Price_USD"] * rate

print(df)
```

OUTPUT

The screenshot shows a Python IDE with two windows. The left window displays the Python code for converting USD prices to INR. The right window shows the output of the code, which is a DataFrame with three rows and three columns: Product, Price_USD, and Price_INR. The output is as follows:

	Product	Price_USD	Price_INR
0	Pen	2	166
1	Book	10	830
2	Bag	25	2075

Conclusion

The program converts prices into Indian Rupees and stores them in a new column for easier understanding.

Question 10: Merge Two CSV Files

Problem Explanation

Sometimes information is stored in multiple files. Merging datasets combines related information using a common key such as Department ID.

Python Code

```
import pandas as pd

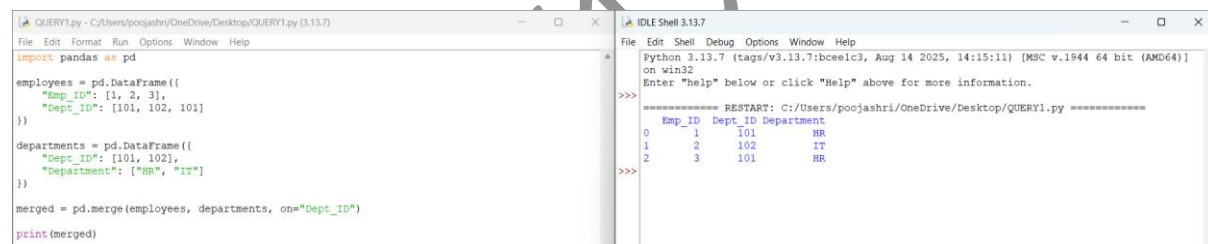
employees = pd.DataFrame({
    "Emp_ID": [1, 2, 3],
    "Dept_ID": [101, 102, 101]
})

departments = pd.DataFrame({
    "Dept_ID": [101, 102],
    "Department": ["HR", "IT"]
})

merged = pd.merge(employees, departments, on="Dept_ID")

print(merged)
```

OUTPUT

A screenshot of a Python IDE window titled 'IDLE Shell 3.13.7'. The left pane shows the Python code for creating two DataFrames and merging them. The right pane shows the output of the code, which is a table with three columns: 'Emp_ID', 'Dept_ID', and 'Department'. The table contains three rows of data.

```
Python 3.13.7 (tags/v3.13.7:bcce1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
   Emp_ID  Dept_ID Department
0        1      101         HR
1        2      102         IT
2        3      101         HR
>>>
```

Conclusion

The program combines employee and department information into one dataset using the common Department ID.

Question 12: Convert CSV into JSON

Problem Explanation

JSON is a popular data format used in web applications. This program converts CSV data into JSON format for easier data exchange.

Python Code

```
import pandas as pd
```

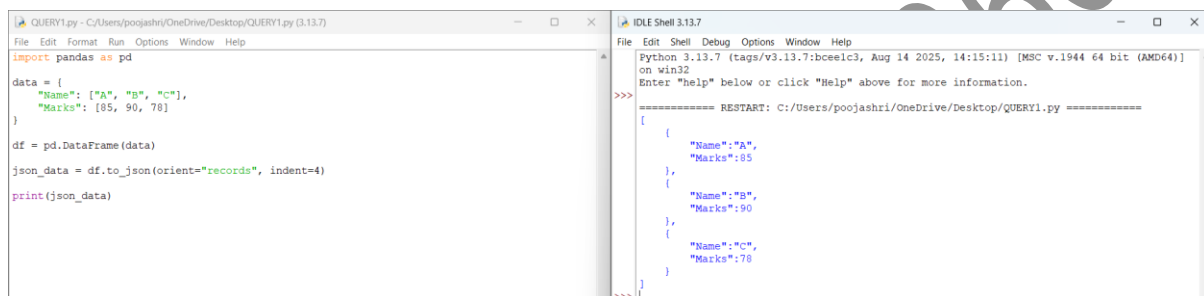
```
data = {
    "Name": ["A", "B", "C"],
    "Marks": [85, 90, 78]
}

df = pd.DataFrame(data)

json_data = df.to_json(orient="records", indent=4)

print(json_data)
```

OUTPUT



```
QUERY1.py - C:/Users/poojashri/OneDrive/Desktop/QUERY1.py (3.13.7)
File Edit Format Run Options Window Help
import pandas as pd

data = {
    "Name": ["A", "B", "C"],
    "Marks": [85, 90, 78]
}

df = pd.DataFrame(data)
json_data = df.to_json(orient="records", indent=4)
print(json_data)
```

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
{
  {
    "Name": "A",
    "Marks": 85
  },
  {
    "Name": "B",
    "Marks": 90
  },
  {
    "Name": "C",
    "Marks": 78
  }
}
>>>
```

Conclusion

The program extracts only the required employee records using filtering conditions.

Question 13: JSON Data Filtering

Problem Explanation

Filtering allows us to display only the required records from a dataset. This program filters employees with salary greater than ₹60,000 and those working in the IT department.

Python Code

```
import pandas as pd

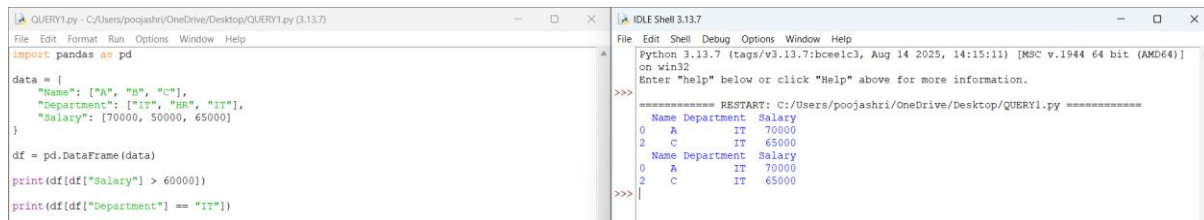
data = {
    "Name": ["A", "B", "C"],
    "Department": ["IT", "HR", "IT"],
    "Salary": [70000, 50000, 65000]
```

```
df = pd.DataFrame(data)

print(df[df["Salary"] > 60000])

print(df[df["Department"] == "IT"])
```

OUTPUT



```

File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
      Name Department  Salary
0      A           IT   70000
2      C           IT   65000
      Name Department  Salary
0      A           IT   70000
2      C           IT   65000
>>>

```

Conclusion

The program extracts only the required employee records using filtering conditions.

Question 16: XML Data Analysis

Problem Explanation

XML files store structured information. This program reads XML data and displays the total number of books, books published after 2020, and books costing more than ₹500.

Python Code

```
import pandas as pd

df = pd.DataFrame({
    "Book": ["A", "B", "C"],
    "Year": [2019, 2022, 2021],
    "Price": [450, 600, 750]
})

print("Total Books:", len(df))
```

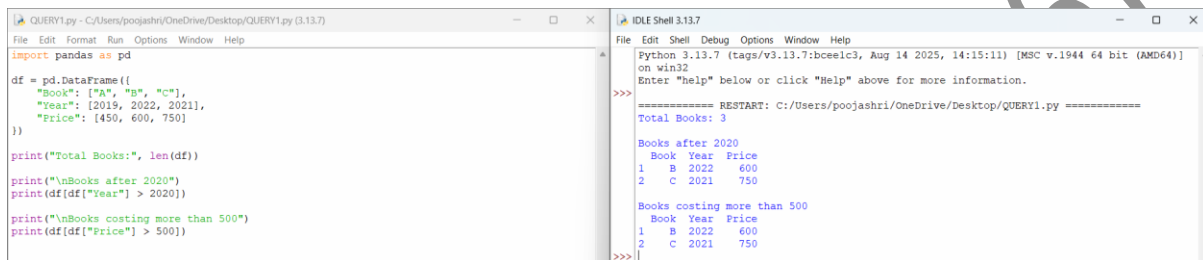
```
print("\nBooks after 2020")
```

```
print(df[df["Year"] > 2020])
```

```
print("\nBooks costing more than 500")
```

```
print(df[df["Price"] > 500])
```

OUTPUT

A screenshot of a Python IDE with two windows. The left window, titled 'QUERY1.py', contains the following code:

```
import pandas as pd

df = pd.DataFrame({
    "Book": ["A", "B", "C"],
    "Year": [2019, 2022, 2021],
    "Price": [450, 600, 750]
})

print("Total Books:", len(df))
print("\nBooks after 2020")
print(df[df["Year"] > 2020])

print("\nBooks costing more than 500")
print(df[df["Price"] > 500])
```

The right window, titled 'IDLE Shell 3.13.7', shows the output of the code:

```
>>>
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====
Total Books: 3

Books after 2020
  Book Year Price
1    B  2022   600
2    C  2021   750

Books costing more than 500
  Book Year Price
1    B  2022   600
2    C  2021   750
>>>
```

Conclusion

The program analyzes book information by counting records and applying filtering conditions.

Question 18: Dictionary-Based Analysis

Problem Explanation

Python dictionaries store data in key-value pairs. This program stores employee salaries, identifies the highest and lowest salary, and calculates the average salary.

Python Code

```
employees = {
    "Rahul": 50000,
    "Anita": 70000,
    "Vijay": 60000
}
```

```
highest = max(employees, key=employees.get)
```



```
lowest = min(employees, key=employees.get)
```

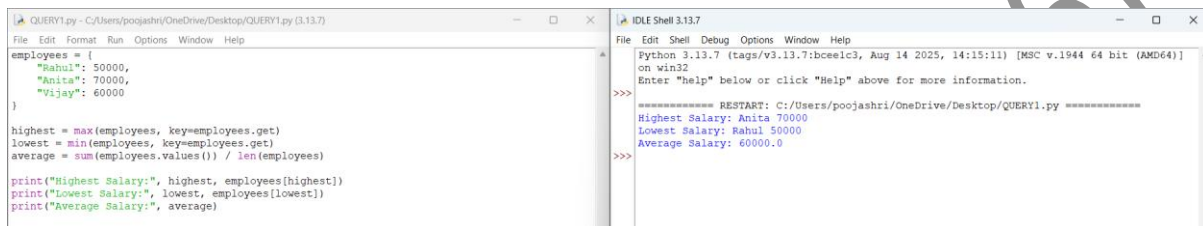
```
average = sum(employees.values()) / len(employees)
```

```
print("Highest Salary:", highest, employees[highest])
```

```
print("Lowest Salary:", lowest, employees[lowest])
```

```
print("Average Salary:", average)
```

OUTPUT

The image shows a screenshot of a Python IDE with two windows. The left window, titled 'QUERY1.py', contains the following code:

```
employees = {  
    "Rahul": 50000,  
    "Anita": 70000,  
    "Vijay": 60000  
}  
  
highest = max(employees, key=employees.get)  
lowest = min(employees, key=employees.get)  
average = sum(employees.values()) / len(employees)  
  
print("Highest Salary:", highest, employees[highest])  
print("Lowest Salary:", lowest, employees[lowest])  
print("Average Salary:", average)
```

The right window, titled 'IDLE Shell 3.13.7', shows the output of the program:

```
>>>  
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]  
on win32  
Enter "help" below or click "Help" above for more information.  
  
===== RESTART: C:/Users/poojashri/OneDrive/Desktop/QUERY1.py =====  
Highest Salary: Anita 70000  
Lowest Salary: Rahul 50000  
Average Salary: 60000.0  
>>>
```

Conclusion

The program uses dictionaries to efficiently store employee information and perform salary analysis such as finding the highest, lowest, and average salary.